



EECS 453

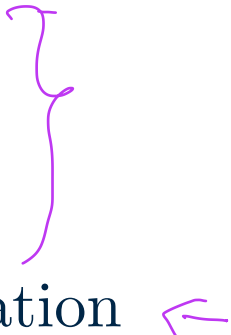
Principles of

Machine Learning

Lecture 5 – Optimization for Machine Learning

Instructor: Prof. Qing Qu

Lecture Agenda

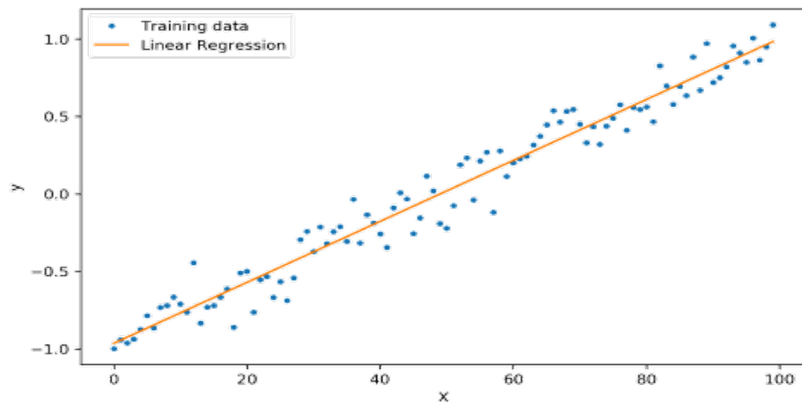
- Basics & Motivating Examples
 - Convexity & Convex Loss
 - Unconstrained Optimization
 - Gradient Descent
 - Stochastic Optimization Methods
 - Newton's Method
 - Theory of Constrained Optimization
- 

Lecture Agenda

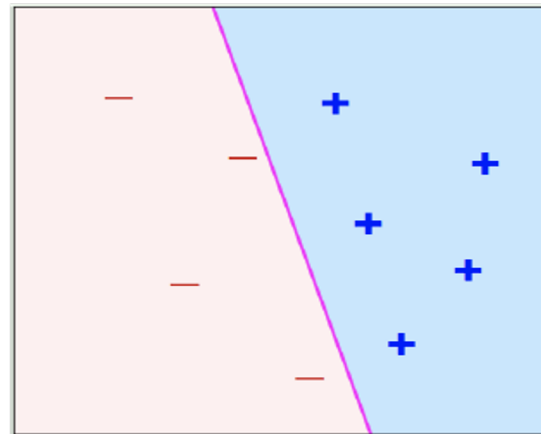
- **Basics & Motivating Examples**
- Convexity & Convex Loss
- Unconstrained Optimization
 - Gradient Descent
 - Stochastic Optimization Methods
 - Newton's Method
- Theory of Constrained Optimization

Regression vs. Classification

Regression



Classification



- Regression is to fit a *continuous* quantity, $y \in \mathbb{R}$ is continuous
- Classification is to fit a *discrete* labels, $y = \{+1, -1\}$ is categorical

Linear Regression

- Solving the *least squares* problem: given $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{y} \in \mathbb{R}^n$

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

Optimization:

- When $n \geq d$ and $\mathbf{X}$ has full column rank, *closed-form* solution

$$\boldsymbol{\theta}_* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Linear Regression

- Solving the *least squares* problem: given $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{y} \in \mathbb{R}^n$

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \underbrace{\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2}_{L(\boldsymbol{\theta})}$$

Optimization:

- When $n \geq d$ and $\mathbf{X}$ has full column rank, *closed-form* solution

$$\boldsymbol{\theta}_* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- When $n < d$, we run into “overfitting” with *no* unique solution.

- Tikhonov regularization/ridge regression:

$$\min_{\boldsymbol{\theta}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \underbrace{\|\boldsymbol{\Gamma}\boldsymbol{\theta}\|_2^2}_{\lambda \|\boldsymbol{\theta}\|_2^2} \quad \boldsymbol{\Gamma} = \sqrt{\lambda} \mathbf{I} \quad \Rightarrow \quad \boldsymbol{\theta}_* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

- Lasso:

$$\min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \underbrace{\lambda \|\boldsymbol{\theta}\|_1}_{\leftarrow \text{optimization}}$$

Why NOT Closed-form Solutions?

Form least-squares and ridge regression, we have closed-form solutions

$$\theta_{\star} = \left(\mathbf{X}^{\top} \mathbf{X}\right)^{-1} \mathbf{X}^{\top} \mathbf{y} \qquad \theta_{\star} = \left(\mathbf{X}^{\top} \mathbf{X} + \lambda \mathbf{I}\right)^{-1} \mathbf{X}^{\top} \mathbf{y}$$

- **Computing the inverse.** The matrix inversion requires $O(d^3)$ computational complexity and memory, which is expensive for large d .

Why NOT Closed-form Solutions?

Form least-squares and ridge regression, we have closed-form solutions

$$\theta_{\star} = \left(\mathbf{X}^{\top} \mathbf{X}\right)^{-1} \mathbf{X}^{\top} \mathbf{y} \qquad \theta_{\star} = \left(\mathbf{X}^{\top} \mathbf{X} + \lambda \mathbf{I}\right)^{-1} \mathbf{X}^{\top} \mathbf{y}$$

- **Computing the inverse.** The matrix inversion requires $O(d^3)$ computational complexity and memory, which is expensive for large d .
- **Ill-conditioned matrix.** If the condition number of the matrix $\mathbf{X}$

$$\kappa(\mathbf{X}) := \frac{|\sigma_{\max}(\mathbf{X})|}{|\sigma_{\min}(\mathbf{X})|} > 10^4 \quad \mathbf{X} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^{\top} \neq 0$$

is large, it will result in *solution instability* via numerical methods.

$$\mathbf{\Lambda} = \text{diag}(\sigma_{\max}, \dots, \sigma_{\min})$$

Logistic Regression

The logistic regression problem:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} -\mathcal{L}(\boldsymbol{\theta}) = - \sum_{i=1}^n [y_i \log h(\boldsymbol{\theta}^\top \mathbf{x}_i) + (1 - y_i) \log (1 - h(\boldsymbol{\theta}^\top \mathbf{x}_i))]$$

where $h(\boldsymbol{\theta}^\top \mathbf{x}) = \eta(\mathbf{x}_i; \boldsymbol{\theta}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top \mathbf{x}_i)}$

Logistic Regression

The logistic regression problem:

$$\min_{\boldsymbol{\theta}} -\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \mathbf{x}_i - \log(1 + \exp(-\boldsymbol{\theta}^\top \mathbf{x}_i))]$$

Its gradient is given by

$$\begin{aligned}\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) &= \sum_{i=1}^n \nabla_{\boldsymbol{\theta}} \left[(y_i - 1) \boldsymbol{\theta}^\top \mathbf{x}_i - \log(1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}_i}) \right] \\ &= \sum_{i=1}^n \left[(y_i - 1) \mathbf{x}_i + \mathbf{x}_i e^{-\boldsymbol{\theta}^\top \mathbf{x}_i} (1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}_i})^{-1} \right] = \sum_{i=1}^n \mathbf{x}_i (y_i - h(\boldsymbol{\theta}^\top \mathbf{x}_i))\end{aligned}$$

Thus, $\sum_{i=1}^n \mathbf{x}_i (y_i - h(\boldsymbol{\theta}^\top \mathbf{x}_i)) = \mathbf{0}$ gives $d+1$ *nonlinear* equations, but it has *no closed-form* solutions (compared to linear regression)

Logistic Regression

- How can we maximize

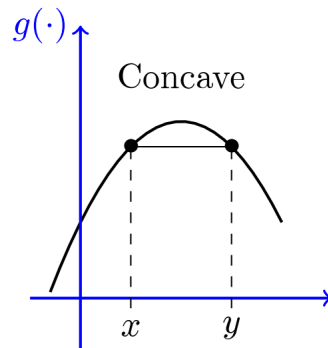
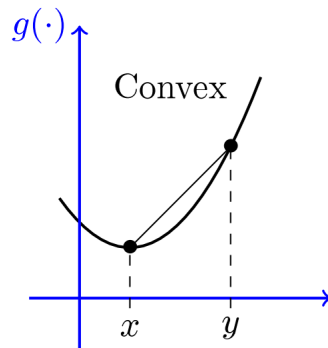
$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \mathbf{x}_i - \log(1 + \exp(-\boldsymbol{\theta}^\top \mathbf{x}_i))]$$

with respect to $\boldsymbol{\theta}$?

- The learning problem:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} -\mathcal{L}(\boldsymbol{\theta})$$

- Bad news: **no** closed-form solution
- Good news: the function is *convex*, we can obtain via **optimization**



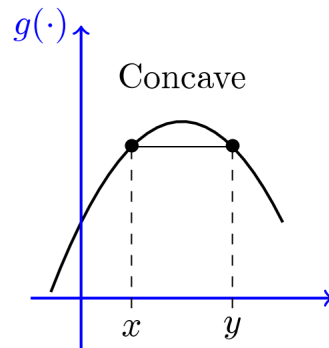
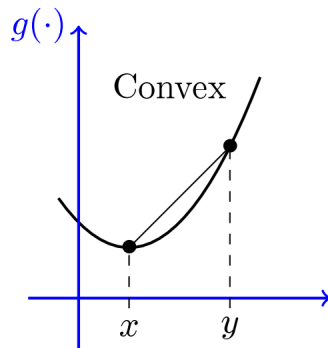
Logistic Regression

The learning problem:

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \mathbb{R}^d} -\mathcal{L}(\theta)$$

where $-\mathcal{L}(\theta)$ is a differentiable *convex function*.

Optimization algorithm: an iterative procedure for solving the above optimization problem



What is an Optimization Problem?

A (mathematical) optimization problem is of the form:

$$\min_{\theta} \mathcal{L}(\theta), \quad \text{s.t.} \quad f_i(\theta) \leq b_i, \quad i = 1, \dots, m$$

- $\theta \in \mathbb{R}^d$ is our **optimization variable**;
 - The problem is **discrete** if θ is a discrete variable;
 - The problem is **continuous** if θ is a continuous variable.

$\theta \in \{0, 1, 2, 3, 4, 5\}$

$\theta \in \mathbb{R}^d$

$\theta \in [0, 1]$

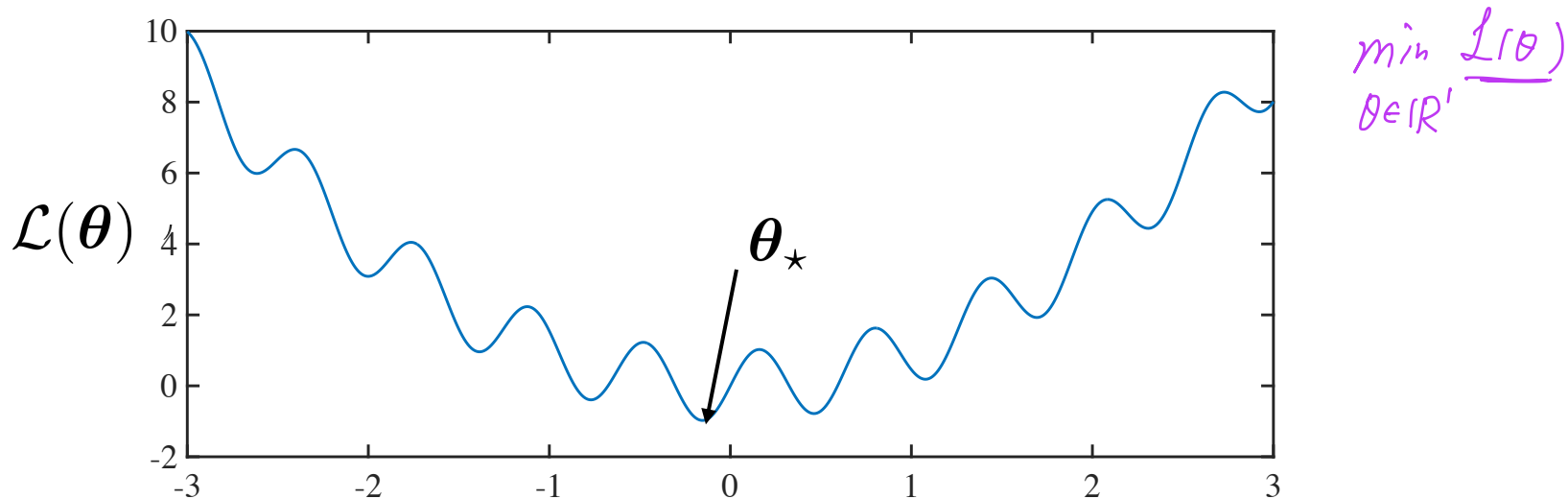
What is an Optimization Problem?

A (mathematical) optimization problem is of the form:

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \quad \text{s.t. } f_i(\boldsymbol{\theta}) \leq b_i, \quad i = 1, \dots, m$$

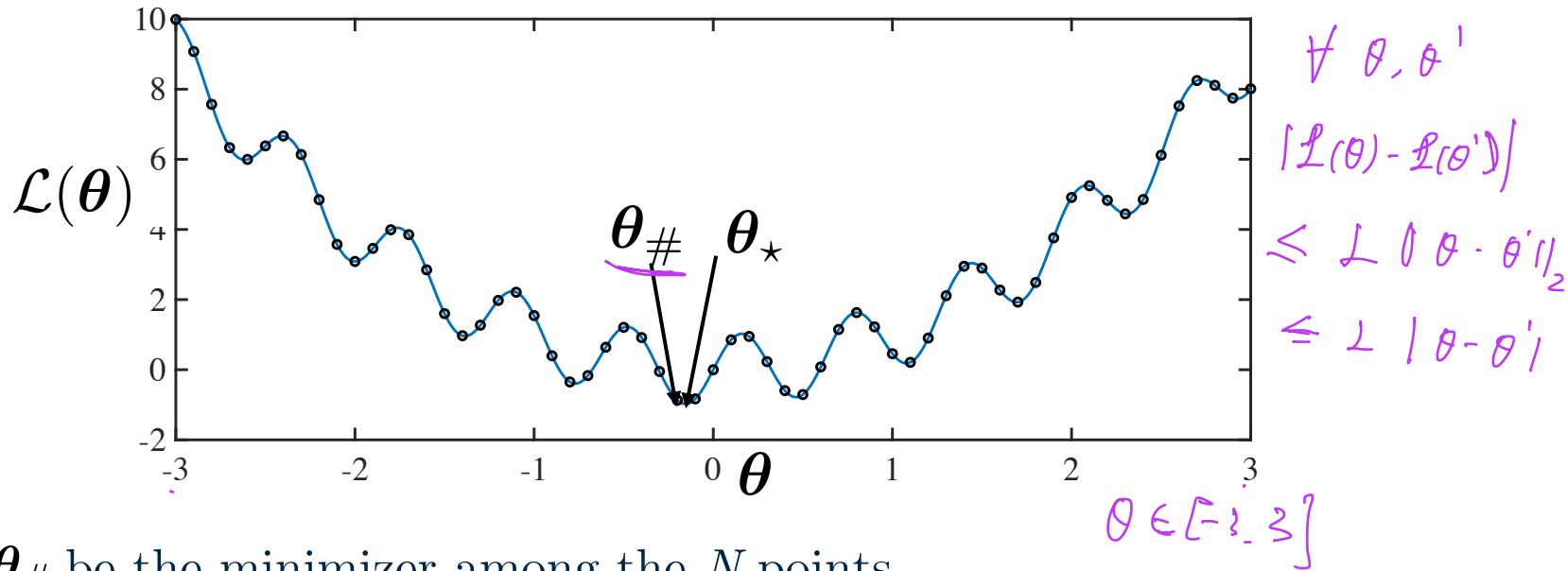
- $\boldsymbol{\theta} \in \mathbb{R}^d$ is our **optimization variable**;
 - The problem is **discrete** if $\boldsymbol{\theta}$ is a discrete variable;
 - The problem is **continuous** if $\boldsymbol{\theta}$ is a continuous variable.
- $\mathcal{L}(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$ is our **objective function**;
- The set $\{\boldsymbol{\theta} \in \mathbb{R}^n \mid f_i(\boldsymbol{\theta}) \leq b_i, i = 1, \dots, m\}$ is our **constraint**.

Why Optimization Algorithms?



- Consider finding the global minimizer θ_*
- A naïve way is to uniformly sample the x-axis N points, and find the minimizer among these N points.

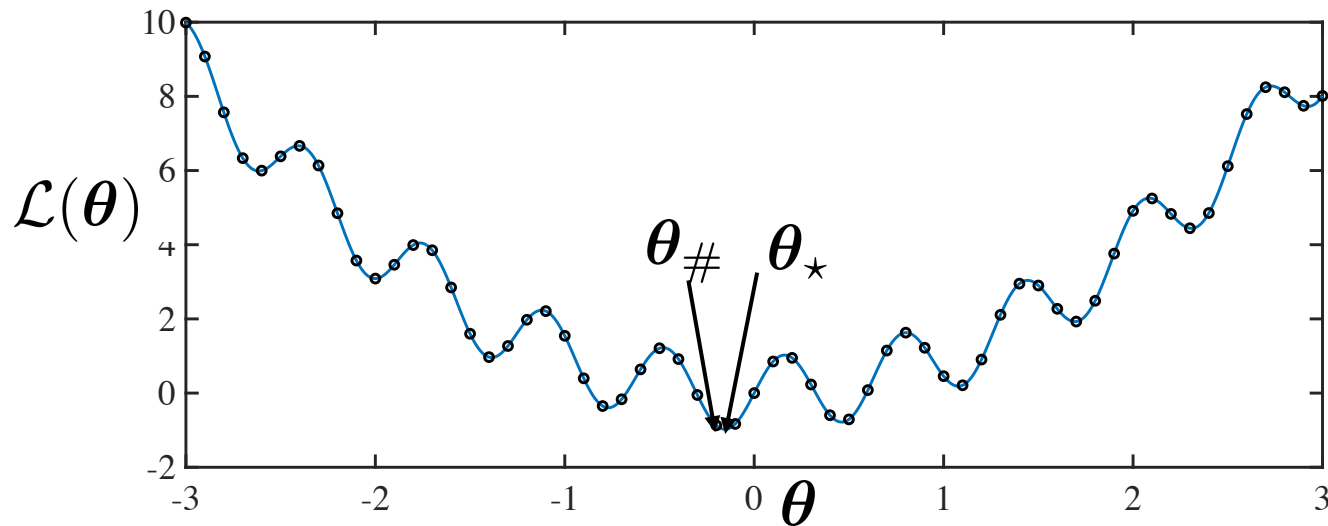
Why Optimization Algorithms?



- Let $\theta_{\#}$ be the minimizer among the N points
- Thus, given the function is L -Lipschitz

$$|\theta_{\#} - \theta_{*}| \leq \frac{6}{N}, \quad |\mathcal{L}(\theta_{\#}) - \mathcal{L}(\theta_{*})| \leq \frac{6}{N} L \leq \epsilon = 10^{-3}, 10^{-4} \quad N \geq \frac{6L}{\epsilon}$$

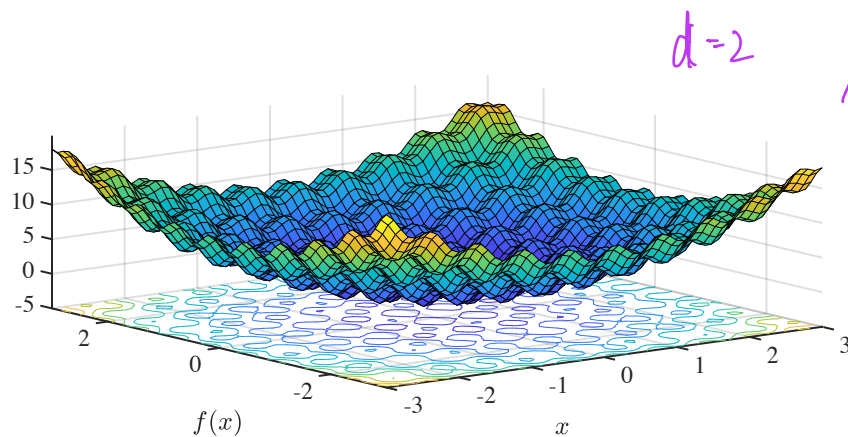
Why Optimization Algorithms?



- Let $\theta_{\#}$ be the minimizer among the N points
- Thus, given the function is L -Lipschitz

$$|\theta_{\#} - \theta_{\star}| \leq \frac{6}{N}, \quad |\mathcal{L}(\theta_{\#}) - \mathcal{L}(\theta_{\star})| \leq \frac{6}{N}L \quad \xrightarrow{N = O(\frac{L}{\varepsilon})} \quad \mathcal{L}(\theta_{\#}) - \mathcal{L}(\theta_{\star}) \leq \varepsilon$$

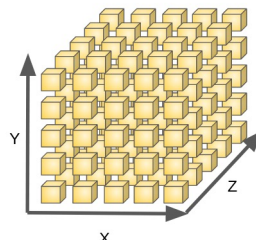
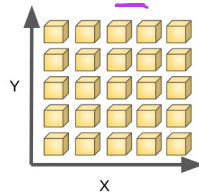
Why Optimization Algorithms? $\xi = 10^{-1}$



$$d=2$$

$$N \geq \frac{6L}{\epsilon^2}$$

$$N \geq \frac{6L}{\epsilon^d}$$



d	1	10	100
$N(\epsilon = 0.1)$	10	10^{10}	10^{100}

- Curse of dimensionality: when $\theta \in \mathbb{R}^d$, then to guarantee

$$\mathcal{L}(\theta_{\#}) - \mathcal{L}(\theta_{\star}) \leq \epsilon$$

we need $N = O(\frac{1}{\epsilon^d})$ samples, growing exponential with d .

Lecture Agenda

- Basics & Motivating Examples
- **Convexity & Convex Loss**
- Unconstrained Optimization
 - Gradient Descent
 - Stochastic Optimization Methods
 - Newton's Method
- Theory of Constrained Optimization

What is Convex Optimization?

Consider the optimization problem:

$$\min_{\theta \in \mathbb{R}^d} \mathcal{L}(\theta) \quad (\mathbf{P})$$

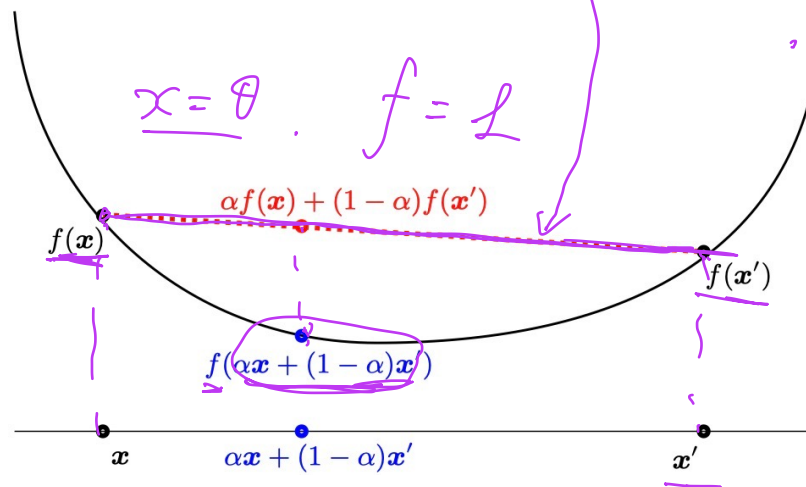
The optimization problem $(\mathbf{P})$ is said to be a *convex problem* iff $\mathcal{L}(\theta)$ is a *convex function*.

Basics: Convex Function and Convex Set

- **Convex function.** A function $\mathcal{L} : \mathbb{R}^d \mapsto \mathbb{R}$ is convex *iff*

$$\checkmark \quad \boxed{\mathcal{L}(\alpha\theta + (1-\alpha)\theta') \leq \alpha\mathcal{L}(\theta) + (1-\alpha)\mathcal{L}(\theta')}$$

for all $\theta, \theta' \in \mathbb{R}^d$ and $\alpha \in (0, 1)$.

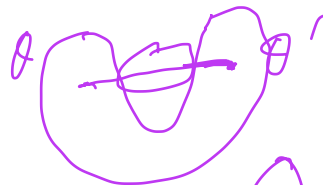


Basics: Convex Function and Convex Set

- **Convex function.** A function $\mathcal{L} : \mathbb{R}^d \mapsto \mathbb{R}$ is convex *iff*

$$\mathcal{L}(\alpha\boldsymbol{\theta} + (1 - \alpha)\boldsymbol{\theta}') \leq \alpha\mathcal{L}(\boldsymbol{\theta}) + (1 - \alpha)\mathcal{L}(\boldsymbol{\theta}')$$

for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathbb{R}^d$ and $\alpha \in (0, 1)$.



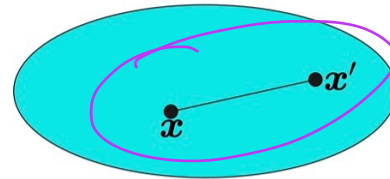
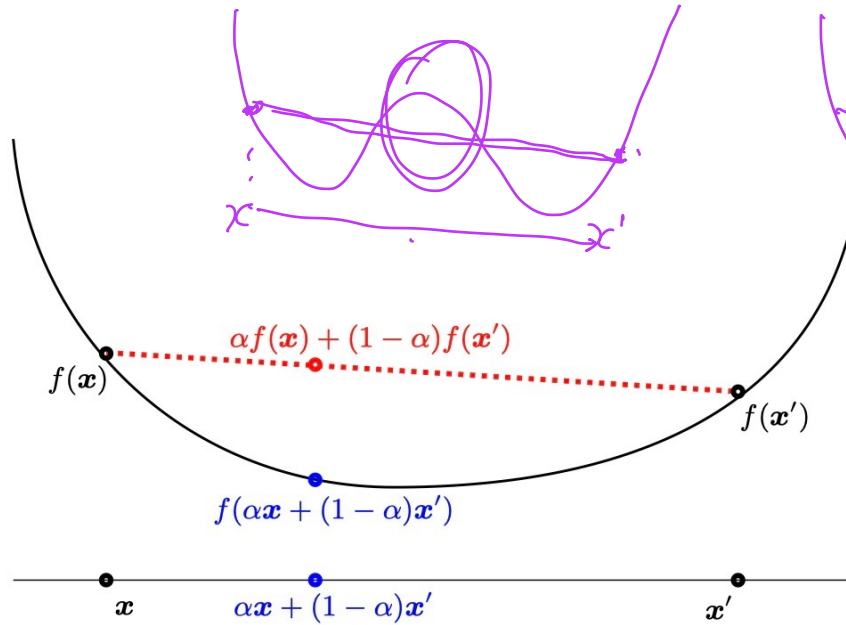
- **Convex set.** Let $\mathcal{C} = \{\boldsymbol{\theta} \in \mathbb{R}^n \mid \mathcal{L}(\boldsymbol{\theta}) - b \leq 0\}$.

$\mathcal{C} \subseteq \mathbb{R}^n$ is a convex set if

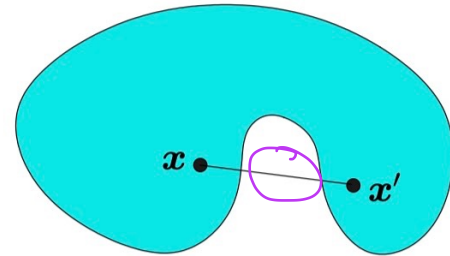
$$\forall \boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathcal{C}, \quad \alpha \in [0, 1], \quad \alpha\boldsymbol{\theta} + (1 - \alpha)\boldsymbol{\theta}' \in \mathcal{C}.$$



Basics: Convex Function and Convex Set



Convex



Nonconvex

Why Convexity ?

The most important property of a convex function:

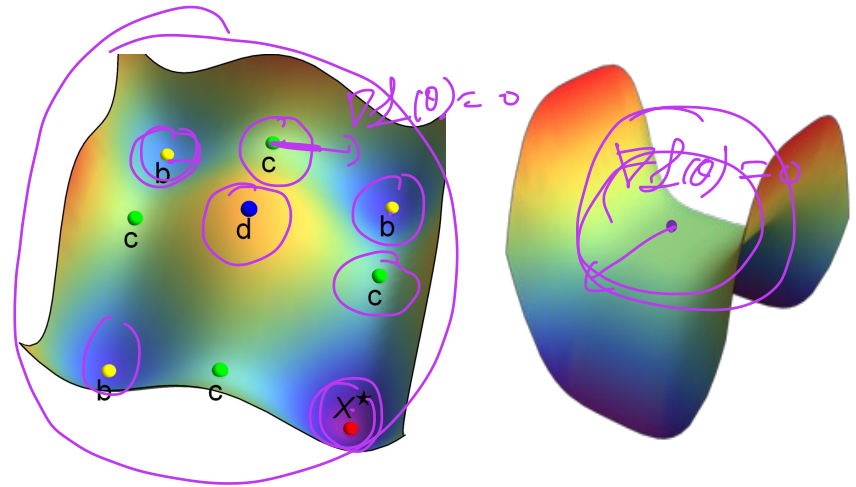
Local Minima are Global Minima

Every *local minimizer* of a convex function over a convex set is also a *global minimizer* of the function over the convex set.

Basics: Critical/Stationary Points

min $\mathcal{L}(\theta)$

- We say θ is a critical/stationary point if $\nabla \mathcal{L}(\theta) = 0$
- A minimizer must be a critical point, but **not** vice versa.
- A critical point could be
 - Local/global minimizer
 - Local/global maximizer
 - Saddle point



Basics: Local Minima vs. Global Minima

Let the function $\mathcal{L} : \mathbb{R}^n \mapsto \mathbb{R}$

min $\mathcal{L}(\theta)$

- **Local minimizer.** We say θ_* is a local minimizer of $\mathcal{L}$, if there exists a scalar $\varepsilon > 0$ such that

$$\mathcal{L}(\theta_*) \leq \mathcal{L}(\theta), \quad \forall \theta \in \mathcal{B}(\theta_*, \varepsilon) := \{z \in \mathbb{R}^n \mid \|z - \theta_*\|_2 \leq \varepsilon\}.$$



Basics: Local Minima vs. Global Minima

Let the function $\mathcal{L} : \mathbb{R}^n \mapsto \mathbb{R}$

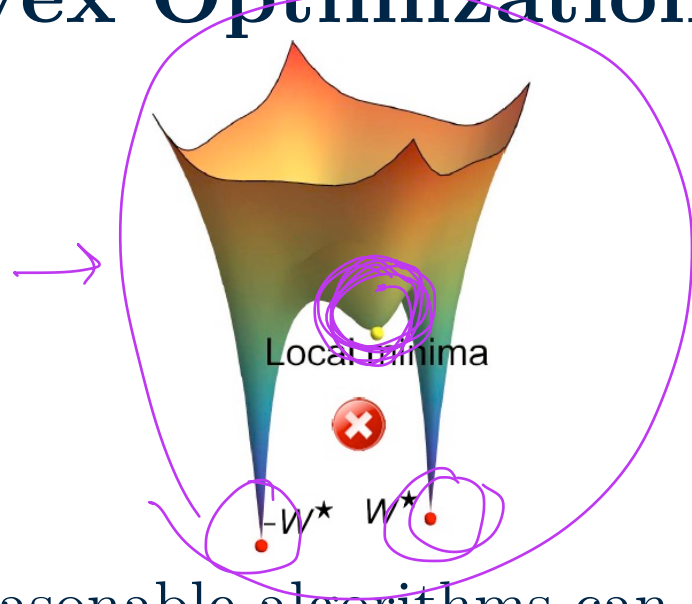
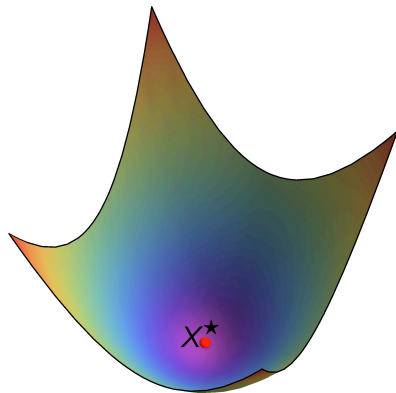
- **Local minimizer.** We say θ_* is a *local minimizer* of $\mathcal{L}$, if there exists a scalar $\varepsilon > 0$ such that

$$\mathcal{L}(\theta_*) \leq \mathcal{L}(\theta), \quad \forall \theta \in \mathcal{B}(\theta_*, \varepsilon) := \{z \in \mathbb{R}^n \mid \|z - \theta_*\|_2 \leq \varepsilon\}.$$

- **Global minimizer.** We say θ_* is a *global minimizer* of $\mathcal{L}$

$$\mathcal{L}(\theta_*) \leq \mathcal{L}(\theta), \quad \forall \theta \in \mathbb{R}^n.$$

Convex vs. Nonconvex Optimization



- **Convex optimization:** Reasonable algorithms can almost always find the global minimum
- **Nonconvex optimization:** In general, find a local minimum is NP-hard.

Why Convexity ?

The **most important** property of a convex function:

Local Minima are Global Minima

Every *local minimizer* of a convex function over a convex set is also a *global minimizer* of the function over the convex set.

How to Check Whether a Function is Convex?

- **By definition:** A function $\mathcal{L}$ is convex *iff*

$$\mathcal{L}(\alpha\boldsymbol{\theta} + (1 - \alpha)\boldsymbol{\theta}') \leq \alpha\mathcal{L}(\boldsymbol{\theta}) + (1 - \alpha)\mathcal{L}(\boldsymbol{\theta}')$$

for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \text{dom } \mathcal{L}$ and $\alpha \in [0, 1]$.

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbf{c}^T \boldsymbol{\theta} + b$$



How to Check Whether a Function is Convex?

- **By definition:** A function $\mathcal{L}$ is convex *iff*

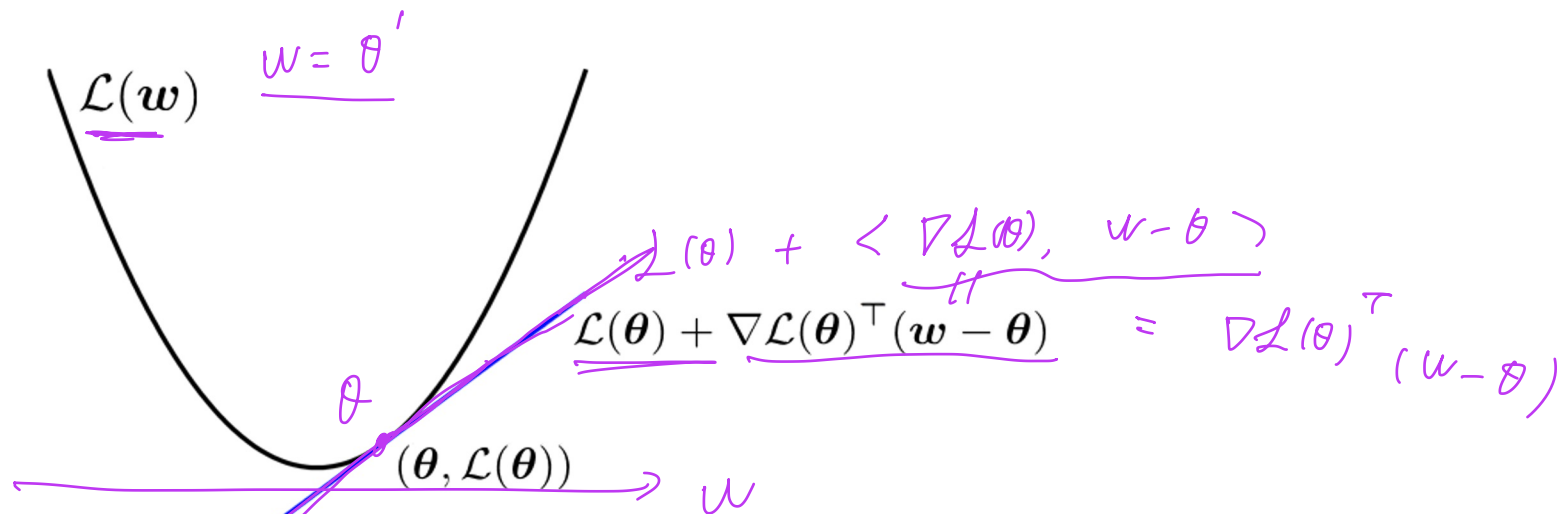
$$\mathcal{L}(\alpha\boldsymbol{\theta} + (1 - \alpha)\boldsymbol{\theta}') \leq \alpha\mathcal{L}(\boldsymbol{\theta}) + (1 - \alpha)\mathcal{L}(\boldsymbol{\theta}')$$

for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \text{dom } \mathcal{L}$ and $\alpha \in [0, 1]$.

- **First-order condition:** if $\mathcal{L}$ is continuously differentiable, the function $\mathcal{L}$ is convex *iff*

$$\mathcal{L}(\boldsymbol{\theta}') \geq \mathcal{L}(\boldsymbol{\theta}) + \langle \nabla \mathcal{L}(\boldsymbol{\theta}), \boldsymbol{\theta}' - \boldsymbol{\theta} \rangle, \quad \forall \boldsymbol{\theta}, \boldsymbol{\theta}' \in \text{dom } \mathcal{L}.$$

First-order Convexity Characterization



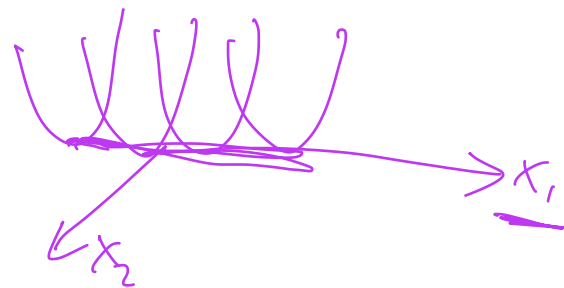
Implication:

$\nabla \mathcal{L}(\theta_\star) = 0$ *if and only if* $\theta_\star$ is a global minimizer.

Gradient and Hessian

- If $\mathcal{L}(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$ is differentiable at $\boldsymbol{\theta}$, its gradient at $\boldsymbol{\theta}$ is

$$\nabla \mathcal{L}(\boldsymbol{\theta}) := \begin{bmatrix} \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_1} \\ \vdots \\ \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_n} \end{bmatrix}$$



- If $\mathcal{L}(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$ is twice differentiable at $\boldsymbol{\theta}$, its Hessian

$$\nabla^2 \mathcal{L}(\boldsymbol{\theta}) := \begin{bmatrix} \frac{\partial^2 \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_1^2} & \cdots & \frac{\partial^2 \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_n \partial \theta_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_1 \partial \theta_n} & \cdots & \frac{\partial^2 \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_n^2} \end{bmatrix}$$

Symmetric matrix

How to Check Whether a Function is Convex?

- **By definition:** A function $\mathcal{L}$ is convex *iff*

$$\mathcal{L}(\alpha\boldsymbol{\theta} + (1 - \alpha)\boldsymbol{\theta}') \leq \alpha\mathcal{L}(\boldsymbol{\theta}) + (1 - \alpha)\mathcal{L}(\boldsymbol{\theta}')$$

for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \text{dom } \mathcal{L}$ and $\alpha \in [0, 1]$.

- **First-order condition:** if $\mathcal{L}$ is continuously differentiable, the function $\mathcal{L}$ is convex *iff*

$$\lambda_i(\nabla^2 \mathcal{L}) \geq 0$$

$$\mathcal{L}(\boldsymbol{\theta}') \geq \mathcal{L}(\boldsymbol{\theta}) + \langle \nabla \mathcal{L}(\boldsymbol{\theta}), \boldsymbol{\theta}' - \boldsymbol{\theta} \rangle, \quad \forall \boldsymbol{\theta}, \boldsymbol{\theta}' \in \text{dom } \mathcal{L}.$$

- **Second-order condition:** if $\mathcal{L}$ is *twice* continuously differentiable, the function $\mathcal{L}$ is convex *iff* $\nabla^2 \mathcal{L} \succeq \mathbf{0}$

$$\nabla^2 \mathcal{L}$$

Examples of Convex Problems

$$X = U \Sigma V^T$$

$$X X^T = U \Sigma V^T V \Sigma U^T \\ = U \Sigma^2 U^T$$

- Linear functions:

$$\mathcal{L}(\theta) = \theta^\top x + c$$

- Least squares:

$$\mathcal{L}(\theta) = \|X\theta - y\|_2^2$$

$$\nabla^2 \mathcal{L}(\theta) = X X^T \succeq 0$$

- Logistic regression:

$$\mathcal{L}(\theta) = - \sum_{i=1}^n [y_i \log h(\theta^\top x_i) + (1 - y_i) \log (1 - \underline{h}(\theta^\top x_i))]$$
$$\underline{\nabla^2 \mathcal{L}(\theta)} \succeq 0$$

Summary: Convex Optimization

- **Benign properties:** every local minimizer is global (no spurious local minimizers)
- **Efficient optimization:** Though usually our loss function does not have closed-form solutions, but we have reliable and efficient algorithms to find the global minimum (next sections)
- **Broad applications:** surprisingly, many engineering problems can give rise to convex optimization (need certain approximations)

Lecture Agenda

- Basics & Motivating Examples
- Convexity & Convex Loss
- **Unconstrained Optimization**
 - Gradient Descent
 - Stochastic Optimization Methods
 - Newton's Method
- Theory of Constrained Optimization

$$\min_{\theta} \underline{J(\theta)}$$

Unconstraint Optimization Problem

Consider the optimization problem:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) \quad (\mathbf{P}) \quad \mathcal{L}(\boldsymbol{\theta}) = \|X\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

We assume that

- The loss $\mathcal{L}(\boldsymbol{\theta})$ is a (convex) function.
- The loss $\mathcal{L}(\boldsymbol{\theta})$ is smooth (continuously differentiable):

$$\nabla \mathcal{L}(\boldsymbol{\theta}) \text{ always exists for all } \boldsymbol{\theta} \in \mathbb{R}^n$$

Solving the Problem via Iterative Algorithm

- Start with an initial point θ_0 , an iterative algorithm $\mathcal{A}(\cdot)$ will generate a sequence of *iterates*

$$\theta_{k+1} = \mathcal{A}(\theta_k), \quad k = 0, 1, 2, \dots$$

- k represents the number of *iteration*
- θ_k represents the *iterate* at k -th iteration

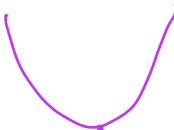
$$\theta_0 \rightarrow \theta_1 \rightarrow \theta_2 \dots \rightarrow \theta_k \rightarrow \theta_{k+1}$$

Solving the Problem via Iterative Algorithm

- Start with an *initial point* θ_0 , an iterative algorithm $\mathcal{A}(\cdot)$ will generate a sequence of *iterates*

$$\theta_{k+1} = \mathcal{A}(\theta_k), \quad k = 0, 1, 2, \dots$$

- k represents the number of *iteration*
 - θ_k represents the *iterate* at k -th iteration
- Iterative descent method:** ideally, we would like

$$\mathcal{L}(\theta_{k+1}) < \mathcal{L}(\theta_k), \quad k = 0, 1, \dots$$


Solving the Problem via Iterative Algorithm

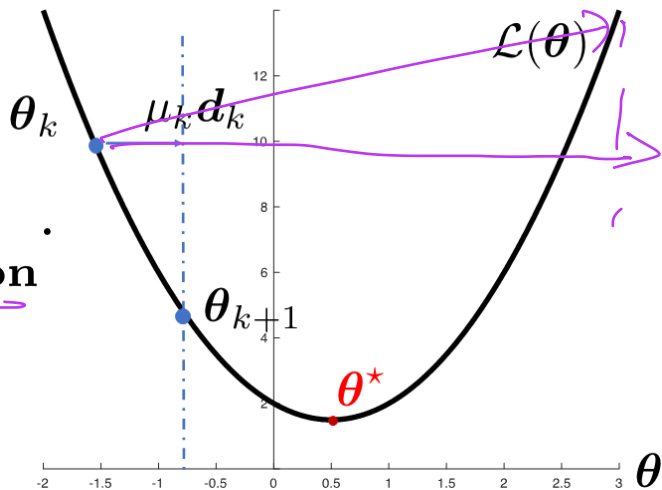
A typical form of $\mathcal{A}(\cdot)$ in practice:

$$\theta_{k+1} = \underbrace{\theta_k}_{\text{current estimate}} + \underbrace{\tau_k}_{\text{stepsize}} \cdot \underbrace{d_k}_{\text{search direction}}$$

- $\tau_k > 0$ is the *stepsize* ✓
- d_k is the *search direction* for

the next step

- The new iterate θ_{k+1} is expected to decrease the function value: $\mathcal{L}(\theta_{k+1}) < \mathcal{L}(\theta_k)$, $k = 0, 1, \dots$



Iterative Descent Algorithms

Start with an initialization θ_0 , and construct a sequence $\{\theta_k\}_{k \geq 1}$, such that

$$\mathcal{L}(\theta_{k+1}) < \mathcal{L}(\theta_k), \quad k = 0, 1, \dots$$

The direction d is said to be a descent direction at θ if

$$\mathcal{L}'(x; d) := \underbrace{\lim_{\tau \searrow 0} \frac{\mathcal{L}(\theta + \tau d) - \mathcal{L}(\theta)}{\tau}}_{\text{direction derivative}} = \nabla \mathcal{L}(\theta)^\top d < 0.$$

$= -\|\nabla \mathcal{L}(\theta)\|_2^2$

$\underline{d} = -\underline{\nabla \mathcal{L}(\theta)}$

What Search Direction d_k Do We Choose?

Proposition (Descent Direction)

Suppose $\mathcal{L}$ is *continuously differentiable*, if there exists a d such that

$$\nabla \mathcal{L}(\theta)^\top d < 0$$

then, there exist a $\tau_0 > 0$ such that

$$\mathcal{L}(\theta + \tau \cdot d) < \mathcal{L}(\theta)$$

for all $\tau \in (0, \tau_0)$. Thus, d is a descent direction at θ .

Lecture Agenda

- Basics & Motivating Examples
- Convexity & Convex Loss
- **Unconstrained Optimization**
 - **Gradient Descent**
 - Stochastic Optimization Methods
 - Newton's Method
- Theory of Constrained Optimization

$$\min_{\theta \in \mathbb{R}^d} \underline{J(\theta)}$$

Gradient Descent Direction

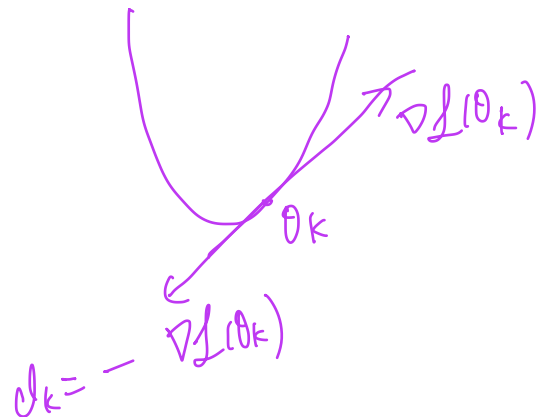
- The proposition tells us: find $\mathbf{d}_k$ satisfies

$$\nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top \mathbf{d}_k < 0$$

then, $\mathbf{d}_k$ must be a descent direction at the current iterate $\boldsymbol{\theta}_k$.

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \tau_k \mathbf{d}_k$$

$$\mathbf{d}_k = -\nabla \mathcal{L}(\boldsymbol{\theta}_k)$$



Gradient Descent Direction

- The proposition tells us: find $\mathbf{d}_k$ satisfies

$$\nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top \mathbf{d}_k < 0$$

then, $\mathbf{d}_k$ must be a descent direction at the current iterate $\boldsymbol{\theta}_k$.

- **Gradient:** the gradient of a *continuously differentiable* function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ at $\boldsymbol{\theta}$ is given by

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \left[\frac{\partial \mathcal{L}}{\partial \theta_1} \quad \cdots \quad \frac{\partial \mathcal{L}}{\partial \theta_d} \right]^\top \in \mathbb{R}^d$$

Gradient Descent Direction

- The proposition tells us: find $\mathbf{d}_k$ satisfies

$$\nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top \mathbf{d}_k < 0$$

then, $\mathbf{d}_k$ must be a descent direction at the current iterate $\boldsymbol{\theta}_k$.

- Gradient descent direction:

$$\mathbf{d}_k = -\nabla \mathcal{L}(\boldsymbol{\theta}_k) \quad \mathbf{d}_k \neq \mathbf{0} \quad \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top \mathbf{d}_k = -\|\mathbf{d}_k\|_2^2 < 0$$

Gradient Descent Method

Gradient descent:

$$\theta_{k+1} = \theta_k - \tau_k \nabla \mathcal{L}(\theta_k)$$

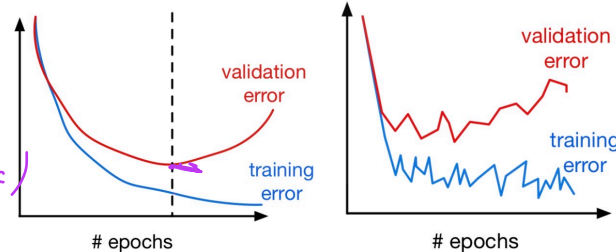
Handwritten annotations: θ_0 above θ_k with a downward arrow, $k=0$ below θ_k , and a purple circle around $\tau_k \nabla \mathcal{L}(\theta_k)$.

The method is also called the *steepest descent*.

Gradient Descent Method

Gradient descent:

$$d_k = -\nabla \mathcal{L}(\theta_k)$$



$$\theta_{k+1} = \theta_k - \tau_k \nabla \mathcal{L}(\theta_k)$$

The method is also called the *steepest descent*.

• Termination Rule:

- Stop when $\|\nabla \mathcal{L}(\theta_k)\|_2$ is small, since $\nabla \mathcal{L}(\theta_*) = 0$ at minimum
- Stop when $\|\theta_{k+1} - \theta_k\|_2$ is small, say $\|\theta_{k+1} - \theta_k\|_2 \leq \varepsilon$
- *In machine learning*, stop when the validation error is going to increase...

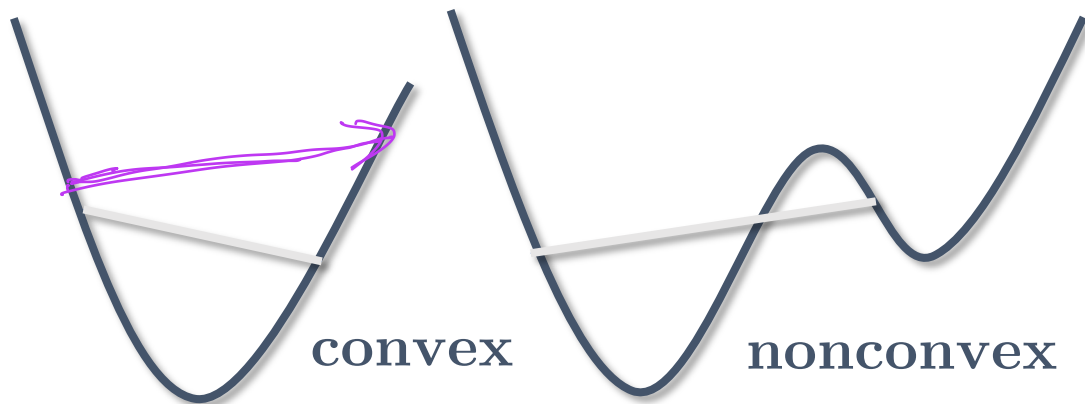
Gradient Descent Method

Gradient descent:

$$\theta_{k+1} = \theta_k - \tau_k \nabla \mathcal{L}(\theta_k)$$

The method is also called the *steepest descent*.

If the function $\mathcal{L}(\theta)$ is **convex**, gradient descent with constant stepsize is guaranteed to converge to **global solutions**.



The Choice of Stepsizes

Gradient descent

$$\theta_{k+1} = \theta_k - \tau_k \nabla \mathcal{L}(\theta_k)$$

$$\tau_k \leq \frac{1}{2L}$$

- The simplest choice for the stepsize is to use a constant stepsize, i.e., $\tau_k = C$ for $k = 0, 1, \dots$

$$\|\nabla \mathcal{L}(\theta) - \nabla \mathcal{L}(\theta')\| \leq L \|\theta - \theta'\|_2, \quad \forall \theta, \theta'$$

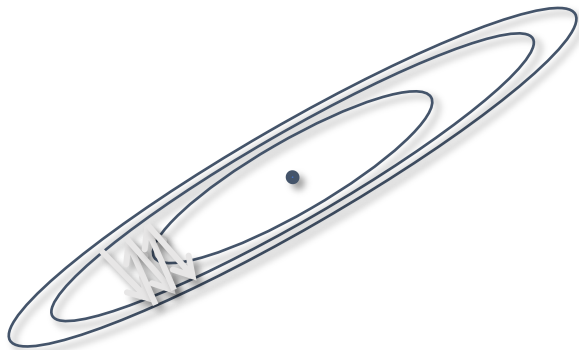
$$\Rightarrow \mathcal{L}(\theta) \text{ is } L\text{-Lipschitz}$$

The Choice of Stepsizes

Gradient descent

$$\theta_{k+1} = \theta_k - \tau_k \nabla \mathcal{L}(\theta_k)$$

- The simplest choice for the stepsize is to use a *constant stepsize*, i.e., $\tau_k = C$ for $k = 0, 1, \dots$
- However constant stepsize is rarely used in practice



The Choice of Stepsizes

Gradient descent

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- The simplest choice for the stepsize is to use a *constant stepsize*, i.e., $\tau_k = C$ for $k = 0, 1, \dots$
- However constant stepsize is rarely used in practice
 - If it is too large, it may lead to divergence
 - If it is too small, bad performance

The Choice of Stepsizes

Gradient descent

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- The simplest choice for the stepsize is to use a *constant stepsize*, i.e., $\tau_k = C$ for $k = 0, 1, \dots$
- However constant stepsize is rarely used in practice
 - If it is too large, it may lead to divergence
 - If it is too small, bad performance

Is there an adaptive rule for choosing the stepsize?

Linear search

Line Search for Stepsize

Gradient descent $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$

The exact line search determine stepsize τ_k by solving

$$\tau_k = \arg \min_{\tau \geq 0} \mathcal{L}(\boldsymbol{\theta}_k - \tau \nabla \mathcal{L}(\boldsymbol{\theta}_k)) \quad \text{exact line search}$$

Line Search for Stepsize

Gradient descent $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$

The exact line search determine stepsize τ_k by solving

$$\tau_k = \arg \min_{\tau \geq 0} \mathcal{L}(\boldsymbol{\theta}_k - \tau \nabla \mathcal{L}(\boldsymbol{\theta}_k)) \quad \text{exact line search}$$

Idea: obtain the most possible descent along direction $-\nabla \mathcal{L}(\boldsymbol{\theta}_k)$

Issue:

- Solving this univariate optimization problem is often not easy...
- Leads to inefficient algorithm

Can we perform an *approximate* linear search?

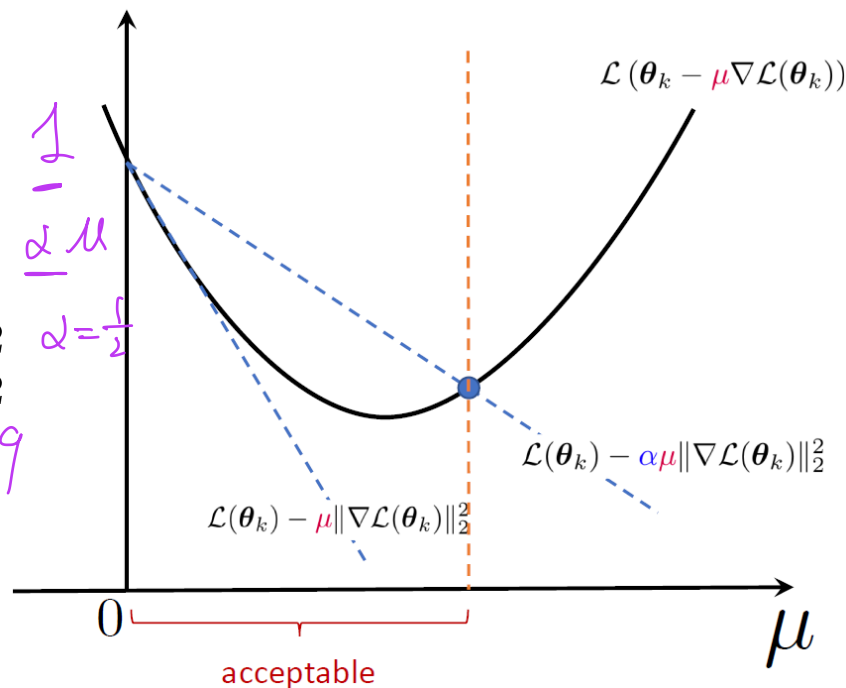
Backtracking Line Search: Armijo Condition

Armijo condition: for some $0 < \alpha < 1$

$$\mathcal{L}(\theta_k - \mu \nabla \mathcal{L}(\theta_k)) < \mathcal{L}(\theta_k) - \alpha \mu \|\nabla \mathcal{L}(\theta_k)\|_2^2$$

Handwritten notes: $\tau = \mu = \frac{1}{2}$, $\mu = \frac{\alpha}{2} \mu$, $\alpha = \frac{1}{2}$, $\alpha = 0.8, 0.9$

- A condition ensuring *sufficient decrease*



Lecture Agenda

- Basics & Motivating Examples
- Convexity & Convex Loss
- **Unconstrained Optimization**
 - Gradient Descent
 - **Stochastic Optimization Methods**
 - Newton's Method
- Theory of Constrained Optimization

Batch, Mini Batch, & Stochastic GD

The objective function in most machine learning problems has the form

(empirical risk/loss) $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(\boldsymbol{\theta})$

parameters $\nearrow$ $\mathcal{L}(\boldsymbol{\theta})$ $\nwarrow$ number of training data n

$\nwarrow$ $\mathcal{L}_i(\boldsymbol{\theta})$ $\nearrow$ Loss function for each training data

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \underbrace{\| \mathbf{X}\boldsymbol{\theta} - \mathbf{y} \|_2^2}_{(\mathbf{x}_i, y_i)} = \frac{1}{n} \sum_{i=1}^n \underbrace{(\mathbf{x}_i^\top \boldsymbol{\theta} - y_i)^2}_{\mathcal{L}_i(\boldsymbol{\theta})}$$

Batch, Mini Batch, & Stochastic GD

The objective function in most machine learning problems has the form

(empirical risk/loss) $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(\boldsymbol{\theta})$

parameters $\mathcal{L}(\boldsymbol{\theta})$

number of training data n

Loss function for each training data $\mathcal{L}_i(\boldsymbol{\theta})$

Batch gradient descent (GD) uses the **entire** data to compute the gradient in each step:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \nabla \mathcal{L}(\boldsymbol{\theta}) \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k}$$
$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla \mathcal{L}_i(\boldsymbol{\theta})$$

$n = 10^6 \sim 10^9$

$(x_i, y_i) \sim D$

Could be very **expensive** for **huge** training dataset!

Batch, Mini Batch, & Stochastic GD

The objective function in most machine learning problems has the form

(empirical risk/loss) $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(\boldsymbol{\theta})$

parameters $\mathcal{L}(\boldsymbol{\theta})$

number of training data n

Loss function for each training data $\mathcal{L}_i(\boldsymbol{\theta})$

1	2	3		n
---	---	---	-------	-----

 i

Stochastic gradient descent (GD) *randomly*

selects **one** sample to compute gradient in each step

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \nabla \mathcal{L}_i(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k}$$

Cheap each iteration
Slow convergence

Batch, Mini Batch, & Stochastic GD

The objective function in most machine learning problems has the form

(empirical risk/loss) $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(\boldsymbol{\theta})$

$\mathcal{L}(\boldsymbol{\theta})$: parameters
 n : number of training data
 $\mathcal{L}_i(\boldsymbol{\theta})$: Loss function for each training data

1,3,7	2,8,16	6,9,15		4,5,n
-------	--------	--------	-------	-------

Mini-batch gradient descent (GD) randomly selects **few** samples to compute gradient in each step

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \tau_k \tilde{\nabla} \mathcal{L}_i(\boldsymbol{\theta}) |_{\boldsymbol{\theta}=\boldsymbol{\theta}_k}$$

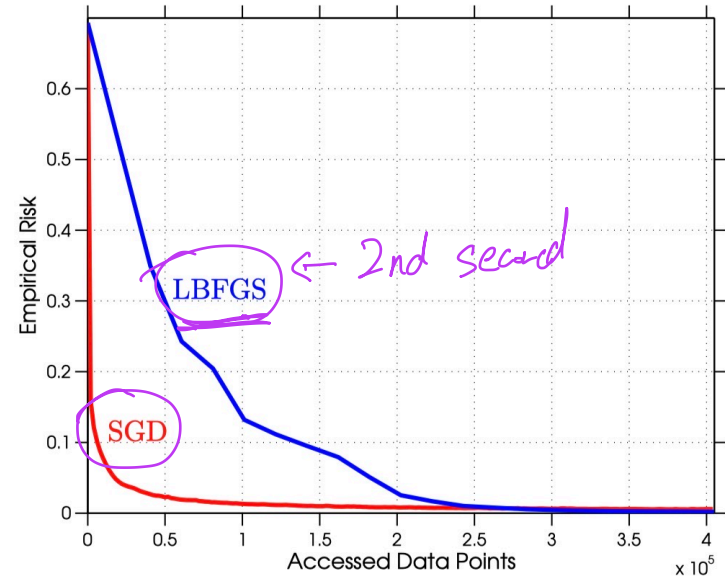
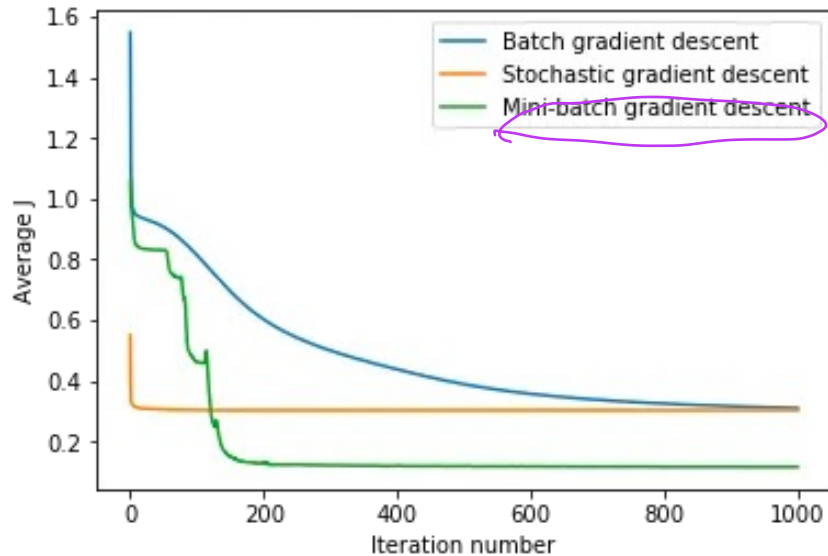
$$\tilde{\nabla} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{|\Omega|_k} \sum_{\ell \in \Omega_k} \nabla \mathcal{L}_\ell(\boldsymbol{\theta})$$

12/ ~ 10 ~ 200



Ω_j

Batch, Mini Batch, & Stochastic GD



<https://adventuresinmachinelearning.com/stochastic-gradient-descent/>

An **epoch** is one **complete** presentation of the *data set*

Lecture Agenda

- Basics & Motivating Examples
- Convexity & Convex Loss
- **Unconstrained Optimization**

- Gradient Descent

- Stochastic Optimization Methods

- **Newton's Method**

} first-order method
: gradient information

← second order method. Hessian

- Theory of Constrained Optimization

Newton's Direction (2nd-order method)

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \quad \text{s.t. } \boldsymbol{\theta} \in \mathbb{R}^n.$$

At the current iterate $\boldsymbol{\theta}_k$, consider the *quadratic* approximation

$$\mathcal{L}(\boldsymbol{\theta}_k + \mathbf{d}) \approx \underbrace{\mathcal{L}(\boldsymbol{\theta}_k) + \mathbf{d}^\top \nabla \mathcal{L}(\boldsymbol{\theta}_k) + \frac{1}{2} \mathbf{d}^\top \nabla^2 \mathcal{L}(\boldsymbol{\theta}_k) \mathbf{d}}_{q_k(\mathbf{d})} \rightarrow \mathcal{L}(\boldsymbol{\theta})$$

2nd-order Taylor expansion of $\boldsymbol{\theta}_k$

Newton's direction:

$$\nabla \mathcal{L}(\boldsymbol{\theta}_k) + \nabla^2 \mathcal{L}(\boldsymbol{\theta}_k) \mathbf{d} = 0$$

$$\mathbf{d} = -(\nabla^2 \mathcal{L}(\boldsymbol{\theta}_k))^{-1} \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

$$\mathbf{d}_k = -(\nabla^2 \mathcal{L}(\boldsymbol{\theta}_k))^{-1} \nabla \mathcal{L}(\boldsymbol{\theta}_k).$$

$$0 = \nabla_{\mathbf{d}} \mathcal{L}(\boldsymbol{\theta}_k + \mathbf{d}) \stackrel{=0}{=} \underbrace{\nabla_{\mathbf{d}} \mathcal{L}(\boldsymbol{\theta}_k)}_{\nabla \mathcal{L}(\boldsymbol{\theta}_k)} + \underbrace{\nabla_{\mathbf{d}} (\mathbf{d}^\top \nabla \mathcal{L}(\boldsymbol{\theta}_k))}_{\frac{1}{2} \nabla (\mathbf{d}^\top \nabla^2 \mathcal{L}(\boldsymbol{\theta}_k) \mathbf{d})} + \nabla^2 \mathcal{L}(\boldsymbol{\theta}_k) \mathbf{d}$$

Hessian

Newton's Method

$$d_k^N = - (\nabla^2 \mathcal{L}(\theta_k))^{-1} \nabla \mathcal{L}(\theta_k)$$

$$\theta_{k+1} = \theta_k + \underbrace{\tau_k}_{\text{stepsize}} \cdot \underbrace{d_k^N}_{\text{descent direction}}$$

$$\nabla \mathcal{L}(\theta_k) = - \nabla^2 \mathcal{L}(\theta_k) d_k^N$$

- It is a **descent direction** when $\nabla^2 \mathcal{L}(\theta_k) \succ 0$

$$\nabla \mathcal{L}(\theta_k)^\top d_k^N = - (d_k^N)^\top \nabla^2 \mathcal{L}(\theta_k) d_k^N < -c_k \|d_k^N\|_2^2 < 0.$$

$$\nabla \mathcal{L}(\theta_k)^\top d_k^N = - \underbrace{(d_k^N)^\top}_{>0} \underbrace{(\nabla^2 \mathcal{L}(\theta_k))}_{>0} d_k^N < 0$$

Newton's Method

$\min_{\theta \in \mathbb{R}^d} f(\theta)$

$d \sim 10, \sim 100$

$$\theta_{k+1} = \theta_k + \underbrace{\tau_k}_{\text{stepsize}} \cdot \underbrace{d_k^N}_{\text{descent direction}}.$$

- It is a **descent direction** when $\nabla^2 \mathcal{L}(\theta_k) \succ 0$

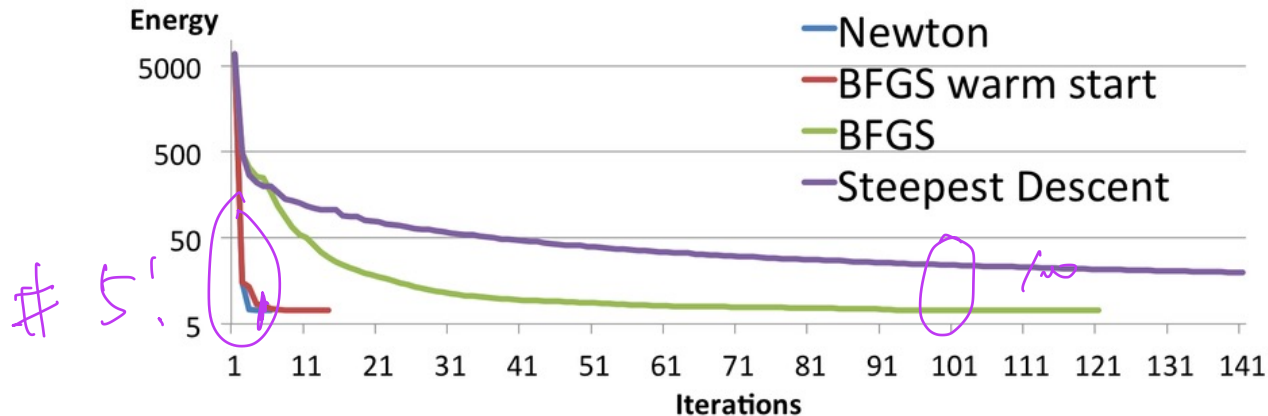
$$\nabla \mathcal{L}(\theta_k)^\top d_k^N = - (d_k^N)^\top \nabla^2 \mathcal{L}(\theta_k) d_k^N < -c_k \|d_k^N\|_2^2 < 0.$$

- The **choice of stepsize**: natural choice is $\tau_k = 1$, but can be adaptively chosen via adaptive linesearch

$$\underline{\mathcal{L}(\theta_k + \alpha d_k^N)} \leq \underline{\mathcal{L}(\theta_k) + \alpha \tau \nabla \mathcal{L}(\theta_k)^\top d_k^N}.$$

Gradient Descent vs Newton's Method

- **Convergence speed:** for strongly convex functions,
 - Gradient descent converges linearly
 - Newton's method has local quadratic converge (much faster)



Newton's Method

$$d_k^N = -(\nabla^2 \mathcal{L}(\theta_k))^{-1} \nabla \mathcal{L}(\theta_k)$$

$$\theta_{k+1} = \theta_k + \underset{\text{stepsize}}{\tau_k} \cdot \underset{\text{descent direction}}{d_k^N}.$$

- **Fast convergence:** Newton's method usually converges much faster than gradient descent (local quadratic vs. linear convergence)
- **Expensive:** requires storing and inverting a large matrix $\nabla^2 \mathcal{L}(\theta)$
- **Unstable:** $\nabla^2 \mathcal{L}(\theta) \succ \mathbf{0}$ might not hold in general

Recipe: Quasi-Newton's Method

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \underbrace{\tau_k}_{\text{stepsize}} \cdot \underbrace{\mathbf{d}_k^{QN}}_{\text{descent direction}}.$$

$$\mathbf{d}_k^{QN} = - \underbrace{\mathbf{H}_k}_{\text{surrogate of } (\nabla^2 \mathcal{L}(\boldsymbol{\theta}_k))^{-1}} \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Design properties of the quasi-Newton matrix $\mathbf{H}_k$:

- It needs to be a good approximation of $(\nabla^2 \mathcal{L}(\boldsymbol{\theta}_k))^{-1}$
- It must be a descent direction with $\mathbf{H}_k \succ \mathbf{0}$
- It can be more efficiently computed than $(\nabla^2 \mathcal{L}(\boldsymbol{\theta}_k))^{-1}$

Gradient Descent vs Newton's Method

- **Convergence speed:** for strongly convex functions,
 - Gradient descent converges linearly
 - Newton's method has local quadratic converge (much faster)
- **Computation complexity:**
 - Because Newton's method needs matrix inversion, the computation is much higher.
- **Solution precision:**
 - Newton's method often reaches higher solution accuracy

Lecture Agenda

- Basics & Motivating Examples
- Convexity & Convex Loss
- Unconstrained Optimization
 - Gradient Descent
 - Stochastic Optimization Methods
 - Newton's Method
- **Theory of Constrained Optimization** ✓

Constrained Optimization

A general constrained optimization problem has the form:

$$\min_{\theta} f(\theta), \quad \text{s.t.} \quad \begin{array}{ll} g_i(\theta) \leq 0, & i = 1, \dots, m, \\ h_i(\theta) = 0, & i = 1, \dots, p \end{array}$$

Handwritten notes in purple:
- A circle around $f(\theta)$ with an arrow pointing to θ .
- A circle around m .
- A circle around p .
- A circle around θ in the minimization term, with $\theta \in \mathbb{R}^d$ written below it.
- Underlines under the constraint rows.

Constrained Optimization

A general constrained optimization problem has the form:

$$\min_{\boldsymbol{\theta}} \underline{f(\boldsymbol{\theta})}, \quad \text{s.t.} \quad \begin{cases} g_i(\boldsymbol{\theta}) \leq 0, & i = 1, \dots, m, \\ h_i(\boldsymbol{\theta}) = 0, & i = 1, \dots, p \end{cases}$$

- **Objective function:** $f(\boldsymbol{\theta}) : \mathbb{R}^d \mapsto \mathbb{R}$
- **Feasibility:** if $\boldsymbol{\theta}$ satisfies all the constraints, $\boldsymbol{\theta}$ is *feasible*
- We assume that $f(\boldsymbol{\theta})$ is well-defined for all feasible $\boldsymbol{\theta} \in \mathbb{R}^d$

A Simple Example

$$\begin{aligned} \min_{\theta \in \mathbb{R}^2} & \quad (\theta_1 - 2)^2 + (\theta_2 - 1)^2 \\ \text{s.t.} & \quad \theta_1^2 - \theta_2 \leq 0 \\ & \quad \theta_1 + \theta_2 \leq 2 \end{aligned}$$

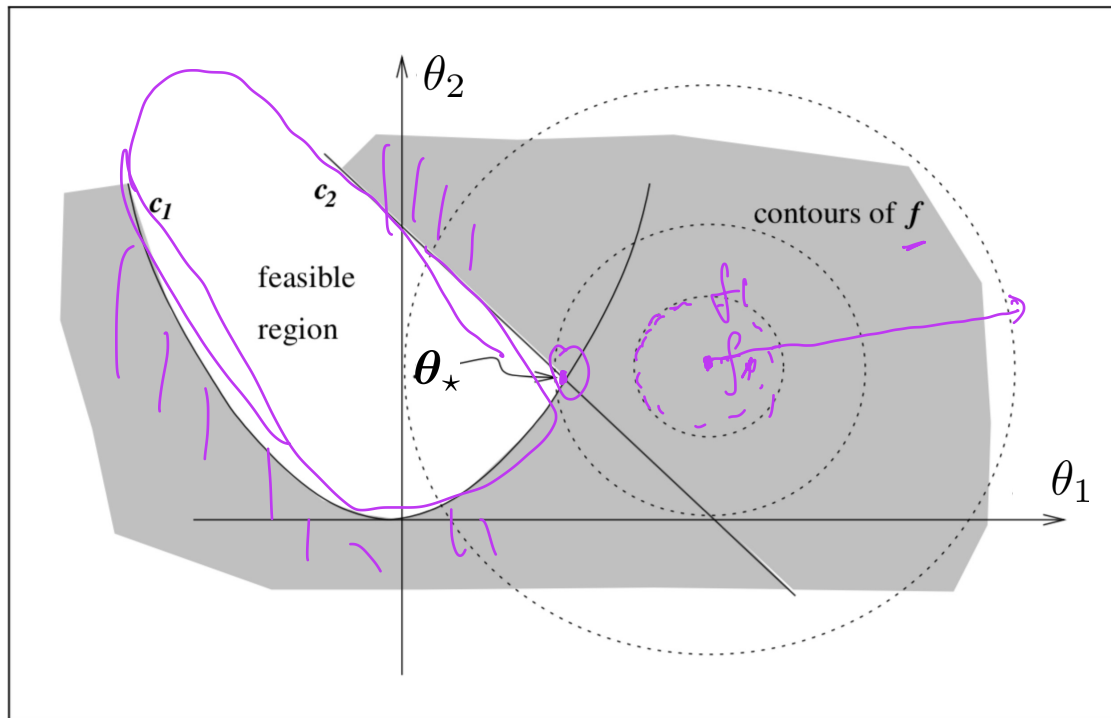
Handwritten notes:

- $f(\theta) = (\theta_1 - 2)^2 + (\theta_2 - 1)^2$
- $\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$
- $g_1(\theta) = \theta_1^2 - \theta_2$
- $g_2(\theta) = \theta_1 + \theta_2 - 2$

We identify:

- The objective variables
- The objective function
- The constraints

A Simple Example



The Lagrangian Function

We can convert a **constrained** problem into an **unconstrained** one via the so-called *Lagrangian function*

$$\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}), \quad \text{s.t.} \quad g_i(\boldsymbol{\theta}) \leq 0, \quad i = 1, \dots, m,$$
$$h_i(\boldsymbol{\theta}) = 0, \quad i = 1, \dots, p$$

The Lagrangian Function

We can convert a **constrained** problem into an **unconstrained** one via the so-called *Lagrangian function*

$$\min_{\boldsymbol{\theta}} \underbrace{f(\boldsymbol{\theta})}, \quad \text{s.t.} \quad \underbrace{g_i(\boldsymbol{\theta}) \leq 0}, \quad i = 1, \dots, m, \\ \underbrace{h_i(\boldsymbol{\theta}) = 0}, \quad i = 1, \dots, p$$

In this case, the Lagrangian function is given by

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu}) := \underbrace{f(\boldsymbol{\theta})}_{f(\boldsymbol{\theta})} + \sum_{i=1}^m \underbrace{\lambda_i g_i(\boldsymbol{\theta})}_{\leq 0} + \sum_{i=1}^p \underbrace{\nu_i h_i(\boldsymbol{\theta})}_{\approx 0}, \quad \underbrace{\lambda_i \geq 0}_{\nu_i \in \mathbb{R}}$$

The Lagrangian Function

We can convert a **constrained** problem into an **unconstrained** one via the so-called *Lagrangian function*

$$\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}), \quad \text{s.t.} \quad g_i(\boldsymbol{\theta}) \leq 0, \quad i = 1, \dots, m,$$
$$h_i(\boldsymbol{\theta}) = 0, \quad i = 1, \dots, p$$

In this case, the Lagrangian function is given by

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu}) := f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}), \quad \lambda_i \geq 0.$$

Here, $(\boldsymbol{\lambda}, \boldsymbol{\nu})$ are called the *Lagrangian multipliers*.

Lagrangian Duality

The Lagrangian provides a way to *bound or solve* the original constrained problem via an alternative unconstrained problem.

Lagrangian Duality

The Lagrangian provides a way to *bound or solve* the original constrained problem via an alternative unconstrained problem.

- If we have the Lagrangian function

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu}) := f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}), \quad \lambda_i \geq 0.$$

$\boldsymbol{\lambda} = [\lambda_1 \ \cdots \ \lambda_m]^\top$ and $\boldsymbol{\nu} = [\nu_1 \ \cdots \ \nu_p]^\top$ are called Lagrange multipliers or dual variables.

Lagrangian Duality

The Lagrangian provides a way to *bound or solve* the original constrained problem via an alternative unconstrained problem.

- If we have the Lagrangian function

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu}) := f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}), \quad \lambda_i \geq 0.$$

$\boldsymbol{\lambda} = [\lambda_1 \ \cdots \ \lambda_m]^\top$ and $\boldsymbol{\nu} = [\nu_1 \ \cdots \ \nu_p]^\top$ are called *Lagrange multipliers* or *dual variables*.

- The *Lagrange dual function* is

$$L_D(\boldsymbol{\lambda}, \boldsymbol{\nu}) = \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu})$$

The Primal & Dual Problems

- The primal

➤ The *primal function* is

$$L_P(\theta) := \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu)$$

If θ is feasible

$$L(\theta, \lambda, \nu) \leq f(\theta)$$

$$\max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu) \leq f(\theta)$$

➤ The *primal optimization problem* is

$$\min_{\theta} L_P(\theta) := \min_{\theta} \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu)$$

What is so “Primal” about the Primal?

$$\min_{\boldsymbol{\theta}} L_P(\boldsymbol{\theta}) := \min_{\boldsymbol{\theta}} \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu})$$

- Suppose $\boldsymbol{\theta}$ is feasible $\Leftrightarrow g_i(\boldsymbol{\theta}) \leq 0, \quad h_i(\boldsymbol{\theta}) = 0$

$$L_P(\boldsymbol{\theta}) = \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}) = f(\boldsymbol{\theta})$$

$\lambda_i = 0 \Rightarrow 0$

What is so “Primal” about the Primal?

$$\min_{\boldsymbol{\theta}} L_P(\boldsymbol{\theta}) := \min_{\boldsymbol{\theta}} \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu})$$

- Suppose $\boldsymbol{\theta}$ is feasible

$$L_P(\boldsymbol{\theta}) = \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}) = f(\boldsymbol{\theta})$$

- Suppose $\boldsymbol{\theta}$ is not feasible $\Rightarrow \exists i, h_i(\boldsymbol{\theta}) \neq 0, \text{ or } g_i(\boldsymbol{\theta}) \geq 0$

$$L_P(\boldsymbol{\theta}) = \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}) = +\infty$$

What is so “Primal” about the Primal?

$$\min_{\boldsymbol{\theta}} L_P(\boldsymbol{\theta}) := \min_{\boldsymbol{\theta}} \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu})$$

- Suppose $\boldsymbol{\theta}$ is **feasible**

$$\underline{L_P(\boldsymbol{\theta})} = \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}) = f(\boldsymbol{\theta})$$

- Suppose $\boldsymbol{\theta}$ is **not feasible**

$$\underline{L_P(\boldsymbol{\theta})} = \max_{\boldsymbol{\lambda}, \boldsymbol{\nu}, \boldsymbol{\lambda} \geq \mathbf{0}} f(\boldsymbol{\theta}) + \sum_{i=1}^m \lambda_i g_i(\boldsymbol{\theta}) + \sum_{i=1}^p \nu_i h_i(\boldsymbol{\theta}) = +\infty$$

The primal encompasses the original problem (i.e., they have the same solution), but it is *unconstrained*

The Primal & Dual Problems

- The primal

➤ The *primal function* is $L_P(\theta) := \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu)$

➤ The *primal optimization problem* is

$$p_\star = \min_{\theta} L_P(\theta) = \min_{\theta} \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu)$$

- The dual

➤ The *dual function* is $L_D(\lambda, \nu) = \min_{\theta} L(\theta, \lambda, \nu)$

➤ The *dual optimization problem* is

$$d_\star = \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu)$$

$d_\star = p_\star$

$$p_\star \geq d_\star$$

Are they equivalent?



The Dual Optimization Problem

$$d_{\star} = \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu)$$

- **Interesting fact:** no matter the choice of f , g_i , or h_i , we have:
 $L_D(\lambda, \nu) = \min_{\theta} L(\theta, \lambda, \nu)$ is always *concave*

The Dual Optimization Problem

$$d_{\star} = \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu)$$

- **Interesting fact:** no matter the choice of f , g_i , or h_i , we have:

$$L_D(\lambda, \nu) = \min_{\theta} L(\theta, \lambda, \nu) \text{ is always } \textit{concave}$$

- **Lower bound property:** for any $\lambda \geq 0$ and any ν , we always have

$$\underbrace{L_D(\lambda, \nu) = \min_{\theta} L(\theta, \lambda, \nu)}_{\text{lower bound}} \leq \underbrace{p_{\star}}_{\text{optimal value}}$$

Weak Duality

Theorem (weak duality) We have

$$\begin{aligned} \underline{d}_\star &= \max_{\lambda, \nu, \lambda \geq 0} \underline{L_D(\lambda, \nu)} = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu) \\ &\leq \min_{\theta} L_P(\theta) = \min_{\theta} \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu) = \underline{p}_\star \end{aligned}$$

Weak Duality

Theorem (weak duality) We have

$$\begin{aligned} d_{\star} &= \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu) \\ &\leq \min_{\theta} L_P(\theta) = \min_{\theta} \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu) = p_{\star} \end{aligned}$$

Proof: Let θ be feasible. Then for any λ, ν with $\lambda \geq 0$

$$L(\theta, \lambda, \nu) = f(\theta) + \sum_{i=1}^m \lambda_i g_i(\theta) + \sum_{j=1}^p \nu_j h_j(\theta) \leq f(\theta)$$

$$\begin{aligned} \text{Hence, } L_D(\lambda, \nu) &= \min_{\theta} L(\theta, \lambda, \nu) \leq \min_{\text{feasible } \theta} L(\theta, \lambda, \nu) \leq f(\theta) \\ &\leq \min_{\text{feasible } \theta} f(\theta) = p_{\star} \end{aligned}$$

Weak Duality

$$p_* - d_* \geq 0$$

Theorem (weak duality) We have

$$\begin{aligned} d_* &= \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu) \\ &\leq \min_{\theta} L_P(\theta) = \min_{\theta} \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu) = p_* \end{aligned}$$

Proof: Thus, for any λ, ν with $\lambda \geq 0$, we have

$$\min_{\theta} L(\theta, \lambda, \nu) \leq p_*$$

Since this holds for any λ, ν with $\lambda \geq 0$, we can take the max to obtain

$$d_* = \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu) \leq p_*$$

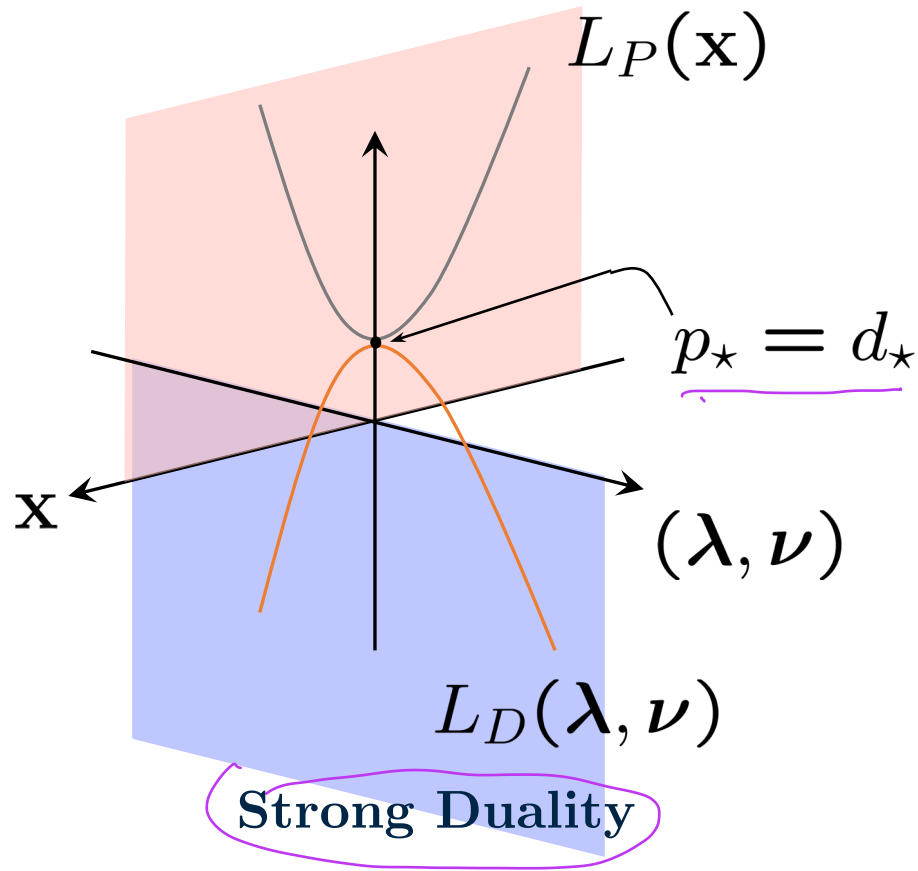
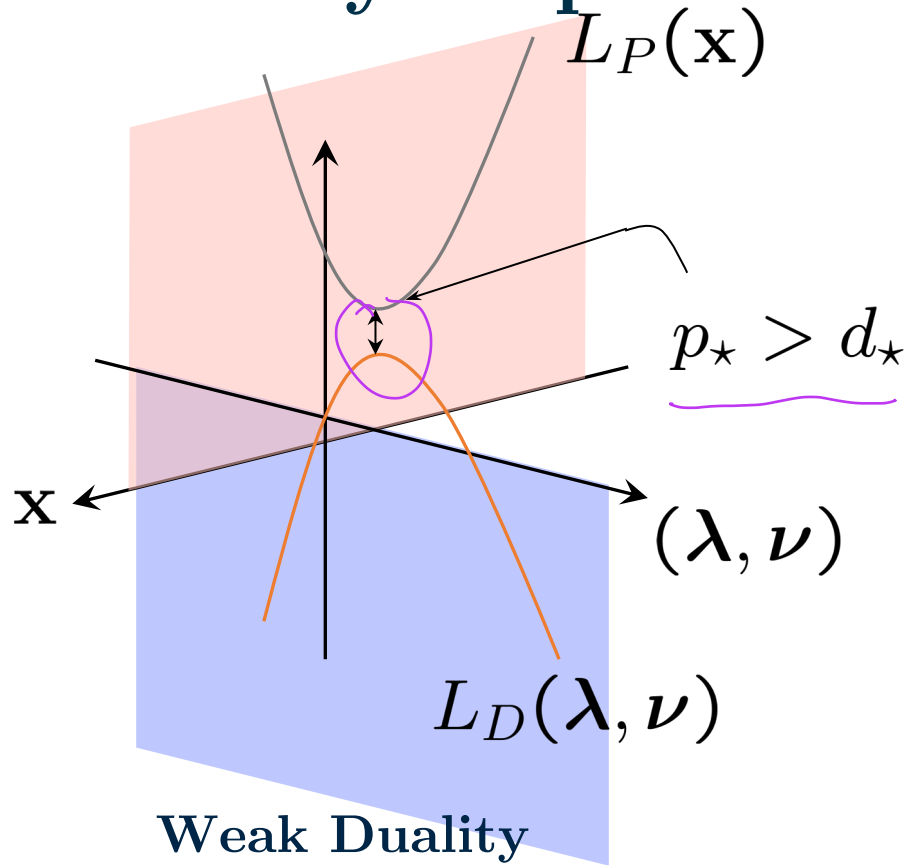
Duality Gap

Theorem (weak duality) We have

$$\begin{aligned} d_{\star} &= \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu) = \max_{\lambda, \nu, \lambda \geq 0} \min_{\theta} L(\theta, \lambda, \nu) \\ &\leq \min_{\theta} L_P(\theta) = \min_{\theta} \max_{\lambda, \nu, \lambda \geq 0} L(\theta, \lambda, \nu) = p_{\star} \end{aligned}$$

- **Duality gap:** the difference $p_{\star} - d_{\star}$ is called the duality gap
 - The duality gap is always nonnegative $p_{\star} - d_{\star} \geq 0$
- **Strong duality:** if $p_{\star} - d_{\star} = 0$, we say that strong duality holds

Duality Gap



KKT Conditions

- Assume that f , g_i , and h_i are all continuously differentiable
- The **Karush-Kuhn-Tucker (KKT)** conditions are the **optimality conditions** that must hold under **strong duality**.

1.
$$\nabla f(\boldsymbol{\theta}_\star) + \sum_{i=1}^m \lambda_i^\star \nabla g_i(\boldsymbol{\theta}_\star) + \sum_{i=1}^p \nu_i^\star \nabla h_i(\boldsymbol{\theta}_\star) = \mathbf{0}.$$

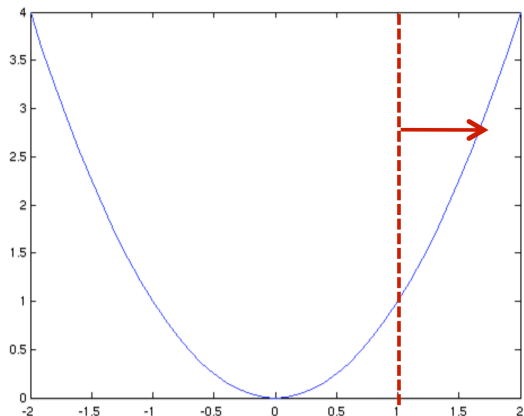
2. **Primal feasibility:**
$$\begin{aligned} g_i(\boldsymbol{\theta}_\star) &\leq 0, & i &= 1, \dots, m \\ h_i(\boldsymbol{\theta}_\star) &= 0, & i &= 1, \dots, p \end{aligned}$$

3. **Dual feasibility:**
$$\lambda_i^\star \geq 0, \quad i = 1, \dots, m$$

4. **Complementary slackness:**
$$\lambda_i^\star \cdot g_i(\boldsymbol{\theta}_\star) = 0, \quad i = 1, \dots, m$$

An Example (Complementary Slackness)

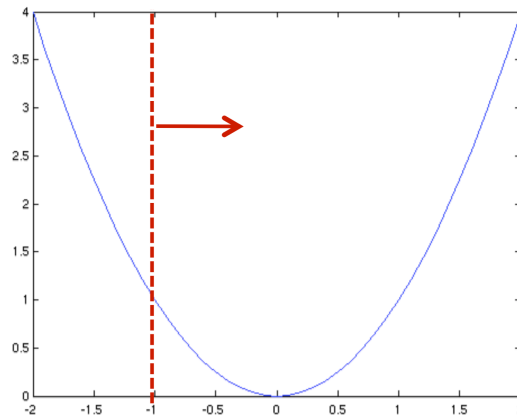
$$\begin{array}{ll}\min_x & x^2 \\ \text{s.t.} & x \geq 1\end{array}$$



$$x^* = 1$$

$\lambda^* = 2 > 0$ constraint is
effective

$$\begin{array}{ll}\min_x & x^2 \\ \text{s.t.} & x \geq -1\end{array}$$



$$x^* = 0$$

$\lambda^* = 0$ constraint is
ineffective

KKT Conditions

- Assume that f , g_i , and h_i are all continuously differentiable
- The **Karush-Kuhn-Tucker (KKT)** conditions are **the optimality conditions** that must hold under **strong duality**.
- The KKT condition will allow us to translate between solutions of the primal and dual optimization problems

$$\theta_{\star} = \arg \min_{\theta} L_P(\theta) \quad \longleftrightarrow \quad (\lambda_{\star}, \nu_{\star}) = \max_{\lambda, \nu, \lambda \geq 0} L_D(\lambda, \nu)$$

Example: Support Vector Machines

$$\begin{aligned} & \min_{\boldsymbol{\theta}, b} \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 \\ \text{s.t. } & y_i (\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

Example: Support Vector Machines

$$\begin{aligned} \min_{\boldsymbol{\theta}, b} \quad & \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 \\ \text{s.t.} \quad & y_i (\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

We have the Lagrangian function:

$$L(\boldsymbol{\theta}, b, \boldsymbol{\lambda}) = \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 + \sum_{i=1}^n \lambda_i [1 - y_i (\boldsymbol{\theta}^\top \mathbf{x}_i + b)]$$

Example: Support Vector Machines

$$\begin{aligned} \min_{\boldsymbol{\theta}, b} \quad & \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 \\ \text{s.t.} \quad & y_i (\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

We have the Lagrangian function:

$$L(\boldsymbol{\theta}, b, \boldsymbol{\lambda}) = \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 + \sum_{i=1}^n \lambda_i [1 - y_i (\boldsymbol{\theta}^\top \mathbf{x}_i + b)]$$

Setting its gradient to zero:

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, b, \boldsymbol{\lambda}) &= \mathbf{0} \\ \frac{\partial}{\partial b} L(\boldsymbol{\theta}, b, \boldsymbol{\lambda}) &= 0 \end{aligned} \quad \Rightarrow \quad \begin{aligned} \boldsymbol{\theta}_\star &= \sum_{i=1}^n \lambda_i^\star y_i \mathbf{x}_i \\ \sum_{i=1}^n \lambda_i^\star y_i &= 0 \end{aligned}$$

Example: Support Vector Machines

$$\begin{aligned} & \min_{\boldsymbol{\theta}, b} \frac{1}{2} \|\boldsymbol{\theta}\|_2^2 \\ \text{s.t. } & y_i (\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

Primal and dual feasibility:

$$y_i (\boldsymbol{\theta}_\star^\top \mathbf{x}_i + b_\star) \geq 1, \quad \lambda_i \geq 0, \quad i = 1, \dots, n$$

Complementary Slackness:

$$\lambda_i^\star \cdot [1 - y_i (\boldsymbol{\theta}_\star^\top \mathbf{x}_i + b_\star)] = 0, \quad i = 1, \dots, n$$

KKT Conditions & Convex Problems

Theorem (strong duality for convex problems)

Suppose that

- the original problem is *convex*, with f and g_i are convex, and h_i are *affine*,
- $(\boldsymbol{\theta}_\star, \boldsymbol{\lambda}_\star, \boldsymbol{\nu}_\star)$ satisfy the **KKT conditions**

Then $\boldsymbol{\theta}_\star$ is primal optimal, $(\boldsymbol{\lambda}_\star, \boldsymbol{\nu}_\star)$ is dual optimal, and the duality gap $p_\star - d_\star$ is *zero*.

KKT Conditions & Convex Problems

Theorem (strong duality for convex problems)

Suppose that

- the original problem is *convex*, with f and g_i are convex, and h_i are *affine*,
- $(\boldsymbol{\theta}_\star, \boldsymbol{\lambda}_\star, \boldsymbol{\nu}_\star)$ satisfy the **KKT conditions**

Then $\boldsymbol{\theta}_\star$ is primal optimal, $(\boldsymbol{\lambda}_\star, \boldsymbol{\nu}_\star)$ is dual optimal, and the duality gap $p_\star - d_\star$ is *zero*.

- For convex problems, KKT condition is a *sufficient condition* for both primal and dual optimality (with **zero duality gap**).

KKT Conditions & Convex Problems

Theorem (strong duality for convex problems)

Suppose that

- the original problem is *convex*, with f and g_i are convex, and h_i are *affine*,
- $(\boldsymbol{\theta}_\star, \boldsymbol{\lambda}_\star, \boldsymbol{\nu}_\star)$ satisfy the **KKT conditions**

Then $\boldsymbol{\theta}_\star$ is primal optimal, $(\boldsymbol{\lambda}_\star, \boldsymbol{\nu}_\star)$ is dual optimal, and the duality gap $p_\star - d_\star$ is *zero*.

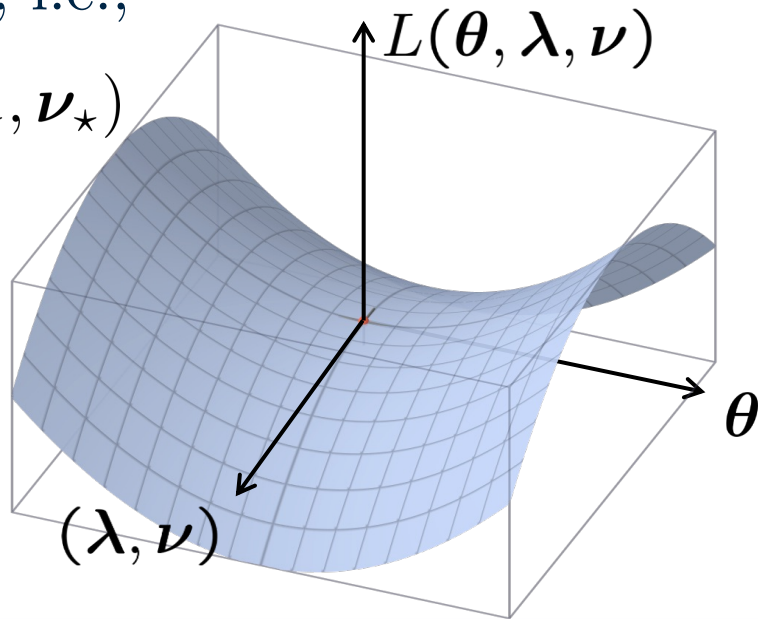
- This also implies that, to solve the primal problem, it is *equivalent* to solve the dual problem.

Saddle Point Property

If $(\boldsymbol{\theta}_*, \boldsymbol{\lambda}_*, \boldsymbol{\nu}_*)$ are primal/dual optimal with zero duality gap,
They are a *saddle point* of $L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\nu})$, i.e.,

$$L(\boldsymbol{\theta}_*, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq L(\boldsymbol{\theta}_*, \boldsymbol{\lambda}_*, \boldsymbol{\nu}_*) \leq L(\boldsymbol{\theta}, \boldsymbol{\lambda}_*, \boldsymbol{\nu}_*)$$

for all $\boldsymbol{\theta} \in \mathbb{R}^d$, $\boldsymbol{\lambda} \in \mathbb{R}_+^m$, and $\boldsymbol{\nu} \in \mathbb{R}^p$.



Summary: KKT Conditions

If a constrained optimization problem is

- differentiable
- convex

then the KKT conditions are necessary and sufficient for primal/dual optimality (with zero duality gap)

Summary: KKT Conditions

If a constrained optimization problem is

- differentiable
- convex

then the KKT conditions are necessary and sufficient for primal/dual optimality (with zero duality gap)

- In this case, we can use the KKT conditions to find a solution to our optimization problem
- If we find $(\boldsymbol{\theta}_*, \boldsymbol{\lambda}_*, \boldsymbol{\nu}_*)$ satisfying the conditions, we have found solutions to both the primal and dual problems

Reference & Further Readings

- *High-Dimensional Data Analysis with Low-Dimensional Models: Principles, Computation, and Applications*. John Wright, Yi Ma. (Appendices B, C, & D)
- *Convex Optimization*. S. Boyd and L. Vandenberghe (2004) (Chapter 3 & 5).